

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/333516254>

A Visual Tool for Supporting Collaborative Code Quality

Conference Paper · May 2019

DOI: 10.1109/CSCWD.2019.8791846

CITATIONS

0

READS

64

5 authors, including:



Matheus Camilo da Silva

University of Trieste

10 PUBLICATIONS 16 CITATIONS

SEE PROFILE



Patricia Rucker de Bassi

Pontifical Catholic University of Paraná

3 PUBLICATIONS 30 CITATIONS

SEE PROFILE



Gregory Moro Puppi Wanderley

Siemens

24 PUBLICATIONS 78 CITATIONS

SEE PROFILE



Cesar Augusto Tacla

Federal University of Technology of Paraná

149 PUBLICATIONS 492 CITATIONS

SEE PROFILE

A Visual Tool for Supporting Collaborative Code Quality

Matheus Camilo da Silva*, Patricia Rucker de Bassi*, Gregory Moro Puppi Wanderley*,
Cesar Augusto Tacla[†] and Emerson Cabrera Paraiso*

*Pontifícia Universidade Católica do Paraná

PPGIA - Graduate Program in Informatics, Curitiba, Brazil

Email: {matheus.camilo, patricia.bassi, gregory, paraiso}@ppgia.pucpr.br

[†]Universidade Tecnológica Federal do Paraná,

CPGEI, Curitiba, Brazil

Email: {tacla}@utfpr.edu.br

Abstract—Organizations have been increasingly relying on software development to manage their business, making it an essential activity. Companies have been using collaborative development environments to speed up their deliveries, resulting in projects tending to be produced by developers from different parts of the world with different cultures. In this context, controlling the software's overall quality by ensuring code quality is a crucial aspect of software development. Software quality can be analyzed based on software parameters like complexity, reusability, testability and maintainability. Such parameters are associated with different group of code metrics which are sets of software measures, providing better insights to developers about their code. In this research we aim to support software developers in better understanding code metrics information. We present a visual tool based on an original method which applies code metrics to an open source collaborative software development environment, grouping them into software product quality parameters. Developers are provided with dashboards and recommendations related to the part of code requiring quality improvement.

Index Terms—Software metrics, code quality, recommendation, dashboard

I. INTRODUCTION

Nowadays, organizations have been increasingly relying on software development to manage their business, making it an essential activity. Companies have been using collaborative development environments to speed up their deliveries. Collaborative software development can be performed in companies displaying different characteristics such as large or small sizes, located in a single location or consisting of several branches over the world, and projects being developed by small or large teams.

Because of such characteristics, software projects tend to be produced collaboratively by developers from different parts of the world with different cultures. In this context, controlling the software's overall quality by ensuring code quality is a crucial aspect of software development. Software companies need to maintain a certain degree of quality on their products. Moreover, software tends to become more complex over a number of releases, making it more difficult to test and maintain, thus decreasing the quality over time [1].

Thus, software quality can be analyzed based on software parameters like complexity, reusability, testability and maintainability. Such parameters are associated with different groups of code metrics, which are sets of software measurements, providing developers with better insights regarding their code [2] [3]. Metrics can help developers to control the code quality, acting as indicators and allowing developers to take actions such as improving the code structure by refactoring it [4] [5] [6].

Reviewing the literature, we found multiple works related to the visualization of code metrics and how they impact on software quality [2]. Moreover, there are also commercial tools that provide such information. However, we found that developers are still facing difficulties to understand and apply the information provided by code quality metrics [7] [8] [9] [10].

In this research we aim to support software developers in better understanding code metrics information. We believe that by better understanding such valuable information, developers can produce source code with a better quality. For instance, they could realise which methods should be changed or more carefully tested, or be attentive to if and else statements. Moreover, as a collaborative team, developers can identify potential quality risks, understand the current state of a project, and track code quality improvement during software development.

In this paper we present a visual tool for supporting developers in better understanding their code's quality. Our tool is based on an original method which applies code metrics to an open source collaborative software development environment, grouping them by software product quality parameters, related to complexity, reusability, testability and maintainability. Then, developers are provided with dashboards for supporting the quality visualization of how their committed code impacts on such software product quality parameters. Moreover, to better support developers regarding code quality, our tool also fetches recommendations for developers, displaying to them information related to the part of code requiring the quality improvement.

We begin the paper by presenting related work. Then, we provide a background of software product quality parameters,

and their relation with code metrics and risk ranges influence on software quality. After that, we present the method our tool is based on, and then we describe the prototype of the tool we built. Next, we provide some discussions and, finally we present conclusions and directions for future work.

II. RELATED WORK

Code metrics have been calculated since the 70s. Some commercial software already calculate and show source code metrics like, Sonar¹, Metrics² for Eclipse³, Understand⁴, even with a visual interface.

We also studied some visual tools from research areas, like: H-Curve [11] that uses a simple visualization method based on Hilbert Curve, displaying the structure of a program, based on software metrics that calculates the complexity of a source code and shows it in a geometrical diagram; Sextant [12] an interactive environment that amplify the information obtained by source code metrics in coloured graphics; The Metric Lens [7] that applied an interactive visualization in which users can specify a wide palette of analysis sorting code metrics on UML diagrams; and CodeMetropolis [13] that is an interactive virtual Minecraft world where source code properties are represented in a graphical way like a game engine. E-Quality [14], also uses code metrics and its risk range influence to assess software quality, representing them in a visual mapping of coupling, size, lack of cohesion and complexity quality attributes.

As a related work research result, we noticed that commercial software tools calculate code metrics, but their visual interface is not attractive, since they focus solely on calculating code metrics. Some tools focus only in one software product quality parameter, like complexity. And others focus on a complex graphic visualization way, that sometimes turn obscure code metrics themselves. E-Quality is the tool that has most similarity with the tool presented here. But none of them provided code practices recommendations to developers.

The visual tool we are proposing uses dashboard concept and grouped code metrics in software product quality parameters as, complexity, testability, maintainability and reusability. The dashboard concept is a clear way to present information with usability. And as a plus, we return recommendations to developers on how (specific directions based on code metrics faults) and where (class/method/package) they can change code in order to increase, or at least maintain, software product quality in a collaborative development environment.

III. SOFTWARE METRICS

The interest in metrics by both academicians and practitioners is growing rapidly. Quality of software product generally depends on complexity, testability, reusability and maintainability parameters. These quality software products parameters seek to manage and reduce structures complexity,

improve maintainability and source code development, and consequently the software itself [14]. All these parameters are associated with certain source code metrics. So, quality of software can be measured by using object-oriented programming metrics. Studies such as [15], [16], [17], [18], [19], [20], [21] [22] and [23] report that source code metrics may indicate code quality parameters such as maintainability, testability, reusability and complexity.

A high complexity in a code can be interpreted as low readability, it means that the code is hard to be handled by developers working on a collaborative team. Some examples of code metrics that influence complexity are the McCabe Cyclomatic (MCC or V(G)) metric, Nested Block Depth (NBD), Lack of Cohesion of Methods (LCOM), and the Response for a Class (RFC) metric.

The MCC measures the number of linearly-independent paths through a program module or method. A high MCC denotes a complex procedure that is hard to understand, test and maintain. The NBD metric shows that more nesting levels make the code difficult to understand, so more complex. The LCOM directly affects the reusability, efficiency and complexity of the software and thus its quality. A high cohesion rate means that classes do not share their attributes with other classes and one can easily reuse them. Classes with a high RFC are more complex and harder to understand, also that, testing and debugging is complicate. In cases of high code complexity, it is important to refactor it, breaking it down into smaller methods or using a design pattern. Other way to reduce complexity is removing any unneeded if or else statements.

Reusability is an important software quality metric, to be able to embed component into various systems, the component must be reusable. Some code metrics that can influence reusability are: Coupling Between Object Classes metric (CBO) calculates the number of classes to which a class is coupled. High CBO is undesirable, since it indicates a design that is difficult to reuse and maintain because of its many interdependencies on other types. WMC (Weighted Methods per Class) metric represents the number of methods defined in a class. Classes with many methods are likely to be more application specific, limiting the possibility of reuse. Also, it can predict how much time and effort is required to develop and maintain a class. So, a high WMC value in a class indicates that developers could restructured it into smaller classes. The Number of Children (NOC) metric calculates the number of immediate sub-classes of a class, measuring the breadth of a class hierarchy. Where maximum DIT (Depth of Inheritance Tree) measures the depth of a class, depth is generally better than breadth, since it promotes reuse of methods through inheritance.

Maintainability represents how easily one can make changes to the software. Some metrics that influence maintainability are: Line of Code (LOC), it indicates the approximate number of lines in the code. A very high count might indicate that a type or method is doing too much work and it might be hard to maintain, so it should be split up. Afferent coupling (Ca) is the number of other packages that depend upon classes within

¹See <https://www.sonarqube.org/> for more information.

²See <http://metrics2.sourceforge.net/> for more information.

³See <https://www.eclipse.org/ide/> for more information.

⁴See <https://scitools.com/features/> for more information.

a package. This value can indicate how changes to classes in a package would influence other parts of the software. Efferent Coupling (Ce) indicates the number of other packages that classes from a package depend upon. It indicates how sensitive a package is for changes to other packages. Code Instability (I) metric is a ratio between Ce and the total package coupling (Ce + Ca). As I tend to 0, indicates a maximally stable package that is completely independent. But as I tend to 1, indicates a totally instable package that has no incoming dependencies and depends upon other packages. CBO, WMC, MCC metrics also influence on software maintainability.

Considering software quality parameter of testability, McCabe is also useful in determining the testability of a method, the higher the MCC value is, the more difficult and riskier the method is to test and maintain. Also, a very high Ca and Ce indicates a code hard to test and maintain, since it has very high complexity and is largely unstable.

Code metrics can influence, not only one, but many software quality metrics, so they can be grouped by their influence considering complexity, maintainability, testability and reusability. In the same way it is possible to provide some recommendations to developers considering code metrics risk levels. By following such recommendations, developers can increase their code quality. Table I shows code metrics grouped by their quality influence on code and Table II shows examples of code metrics and their range of risks.

TABLE I
CODE METRICS QUALITY INFLUENCE

Quality Influence	Code Metric
Complexity	MCC(VG), LCOM*, RFC, MLOC, WMC, DIT, NORM, NOF, NSF, SIX, CBO, RFC
Testability	MCC(VG), Ca, Ce, WMC, NOM, RFC
Reusability	CBO, WMC, DIT, NOC, PAR, NBD, LCOM*, NOM, Ca, Ce, I(RMI), A(RMA)
Maintainability	LOC, CBO, WMC, MCC(VG), NOF, NSF, SIX, Ca, Ce, I(RMI), A(RMA)

TABLE II
SOME CODE METRICS AND THEIR RISK RANGE

Code Metric	Risk Range
Number of Parameters	Good : $PAR \leq 2$ Regular : $2 < PAR \leq 4$ Bad : $PAR > 4$
Nested Block Depth	Good : $NBD \leq 1$ Regular : $1 < NBD \leq 3$ Bad : $NBD > 3$
Method Lines of Code	Good : $MLOC \leq 10$ Regular : $10 < MLOC \leq 30$ Bad : $MLOC > 30$

According to the code metric risk range and the influence it has on the code quality, developers may receive recommendations based on code metrics indicating a bad result. Thus, they can correct their code to increase its quality. Table III shows examples of recommendations related to bad range on code metric.

TABLE III
RECOMMENDATIONS RELATED TO CODE METRIC RISK RANGE EXAMPLE

Metric	Quality Influence	Recommendation
MCC or VG	Complexity	The <method: class: package> has too many decision points (if, then else). It gives a high complexity. Try to divide in smaller problems the whole problem solved by the method. Create other methods and call then inside the original one.
Number of Parameter - PAR	Reusability	The <method: class: package> has too many parameters. It means that it is too specialized and can be difficult to be reused. Reduce the number of parameters, try to pass the data using a user-defined structure or an object.
Nested Block Depth NBD	Reusability	The <method: class: package> is too deep. It appears that there are lot of structures inside other structures, such as a loop, as if statement inside it, then another structure inside the if, and so on. It means that it is too specialized and can be difficult to be reused. Reduce the number of parameters, try to pass the data using a user-defined structure or an object.

Considering that having developers understanding how their manner of coding can influence software quality, and recommendations regarding maintainability, testability, reusability and complexity, available to increase code quality, we present a visualization tool that can help developers to better contribute in a collaborative software development project.

IV. METHOD

In this section we introduce the method our visual tool is based on for supporting code quality comprehension. Fig. 1 shows the proposed method. It consists of the steps of “Extract Quality Metrics,” “Store Versioning System,” “Analyze Complexity,” “Analyze Reusability,” “Analyze Maintainability,” “Analyze Testability,” “Fetch Recommendations,” “Build Dashboard,” “Synthesize Code Quality Information” and “Present Quality Information.”

When a developer of collaborative software development compiles her code, the software quality metrics of Table I (section III) are extracted automatically and in a non-intrusive way. The purpose of the step “Extract Quality Metrics” is to realize such extraction, receiving as input the developer’s code and then outputting the quality metrics.

The next step, named “Store Versioning System,” receives the extracted metric measures along with the developer’s code to store them in a versioning system. The goal is to support the

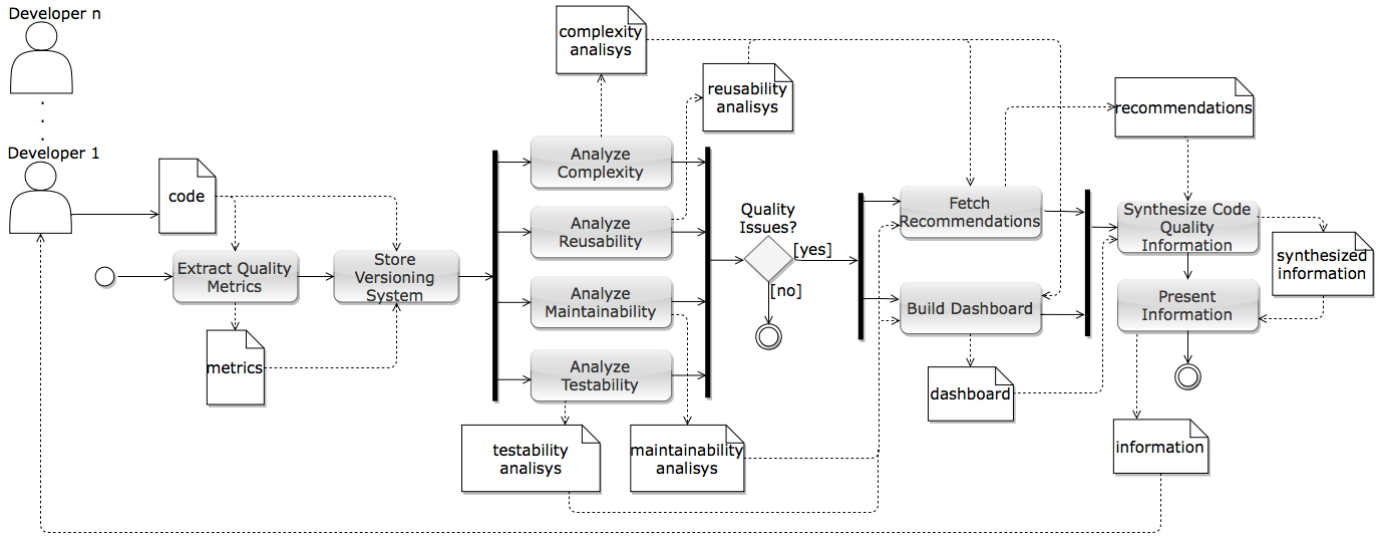


Fig. 1. The proposed method.

collaboration among the team members by organizing better the quality information and the code through such a version management.

After that, the quality metrics are retrieved globally from the versioning system and a number of different analyses are performed. The steps “Analyze Complexity,” “Analyze Reusability,” “Analyze Maintainability” and “Analyze Testability” represent the analyses that are carried out based on the metric measures.

In the “Analyze Complexity” step, our method performs analyses related to the complexity of the code. To do that, it uses the complexity metrics described in Table I.

The “Analyze Reusability” step aims to analyze the reusability of the code. In this step we use the software metrics related to reusability described in Table I.

The “Analyze Maintainability” step analyzed the maintainability of the code. The analysis process uses the software quality metrics related to maintainability described in Table I.

In the “Analyze Testability” step, the method carries out analyses related to the testability of the code. To do that, it uses the testability quality metrics described in Table I.

After the analyses are performed, if there are no quality issues in the code, then the method is finished. Otherwise, the steps “Fetch Recommendations” and “Build Dashboard” are performed in parallel.

The purpose of the “Fetch Recommendations” is to try to fetch code quality recommendations from a database for the issues found. The input of this step is the quality analyses performed in the previous steps of the method, and the output is textual recommendations fetched from the database.

In parallel, the “Build Dashboard” step aims to receive the quality analyses and then build a graphical dashboard. The goal is to build a number charts which will compose the dashboard, in order to provide a clear, understandable and easy way for visualizing the quality information.

Next, the step “Synthesize Code Quality Information” intends to receive the fetched recommendations along with the dashboard to structure them and synthesize all the information. The output of this step is the synthesized information.

The last step of the proposed method, named “Present Quality Information,” aims to take the synthesized information outputted in the previous step and present it to the developer. Note that such an information could also be available to other team members (if desired), for instance, software managers to support the collaboration to improve the code quality. The output of this step and of the proposed method is the information presented to the developer.

V. PROTOTYPE

In this section we present a prototype implementing our visual tool for supporting developers to better understand code quality information. First, we describe the architecture of our prototype, and then after that we show the features it provides for supporting software developers.

A. Architecture

The prototype is composed of two modules, developed in a web service structure. The first module was created using Java, it runs on a web server and it works basically as an API (Application Programming Interface), being capable of sending or receiving data through the REST paradigm and http protocol requests, this module is responsible for the prototype’s business logic, database managing, as well as data validation.

The second module is the client-side aspect of the application, it was created using HTML/CSS and JavaScript. It allows the interaction between the tool and the user by providing intuitive interfaces and a dashboard, designed to present the code metrics statistics and recommendations to the user, it achieves data persistence by making requests to the REST API module.

This tool is designed to be used by one or more developers working on the same project, in accordance with the methodology presented in Figure 1. The projects designed to work with the proposed tool, are meant to be created using java, built with the Java Integrated Development Environment Eclipse and have it's metrics extracted using the plugin Metrics, since the prototype itself does not account for the first two steps of our methodology (extracting and storing the metrics in a versioning platform). It acquires the code metrics by allowing users to synchronize their versioning platform's account, such as GitHub for instance, to the tool's account, enabling the tool to analyze the project's code quality throughout the remaining of our model steps.

B. Features

To provide users with all the information related to the software quality of their projects, the tool presented in this paper is composed of some important features.

As the tool concludes all the analyses of the metrics, it displays the processed information throughout a dashboard, where the developer can assess key performance indicators. As shown in Figure 2.



Fig. 2. Display of quality metrics

Once a chart is selected, the tool may also make recommendations to the developer about their code base. These recommendations are possible to be made by the system through [3]'s work, which associates metric values to code base's characteristics, as shown in Figure 3.

VI. DISCUSSION

The proposed approach supports collaborative software developers with the code quality comprehension in a number of ways. Following we discuss the main advantages and possible limitations of our approach.

After a developer performs a commit, she is aware of the quality of her code in a visual, more intuitive and clear way. By providing a dashboard with several different charts showing the quality information, our approach aims to offer a high level environment for motivating developers to improve their code quality. Moreover, developers also receive more detailed information about quality issues along with the part of the

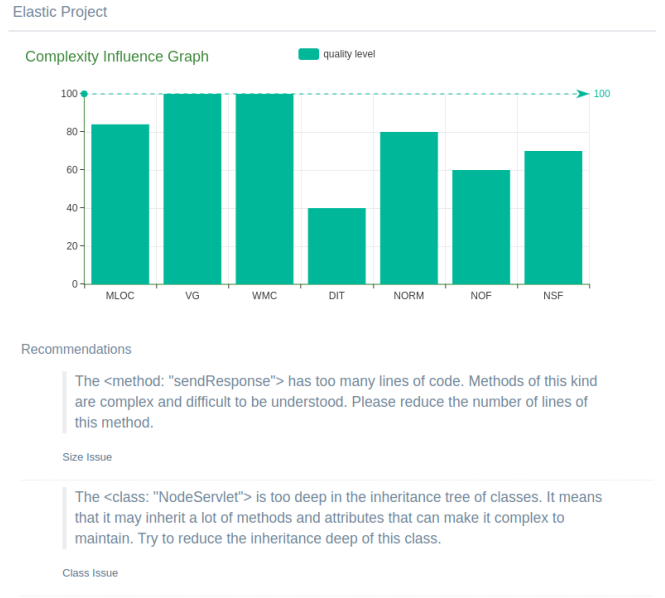


Fig. 3. Complexity recommendations

code with such an issue. In addition, our tool provides quality recommendations for developers for each issue found in their code.

One of the key elements in our approach is the storage of extracted metrics files in a code versioning platform, which facilitate the process of metric analysis, even making possible for it to be made by third parties such as the tool presented. It also contributes to the collaborative development environment by making the metrics a real part of the project.

Through our prototype we could validate our approach for supporting software developers to improve their code quality. The methodology presented was applied to the Elastic's open source project "elasticsearch-transport-ware"⁵, developed collaborative with Java by a community of developers with the platform Github during a 25 months period. The Elastic's project was developed throughout 18 commits, containing 5 classes, 277 lines and a total of 159 metrics. The visual tool was able to analyze and display metrics of complexity, reusability, testability and maintainability, revealing that the project has a few issues with complexity and maintainability, which can be resolved by following the recommendations given by the tool.

However, one should note that we are currently in the early stages of a full evaluation of our prototype with software development teams working in real-world projects. This evaluation is meant to assess the impact of the tool's recommendations in a real-world scenario by measuring the code quality throughout the development of a project in a collaborative professional environment. In the event of a lack of a significant impact or even a negative one, our evaluation plans to analyze if the recommendations are being followed by the developers or if

⁵<https://github.com/elastic/elasticsearch-transport-ware>

they are truly effective. Because such an evaluation requires a considerable amount of time and resources to be performed, for instance, to collect enough data during a certain period from a number of participants working collaboratively, we are designing it carefully.

VII. CONCLUSION AND FUTURE WORK

The increasingly dependency of technology by modern society is compelling companies to practice collaborative software development in order to keep up with demand while also trying to maintain an acceptable quality of code throughout projects developed by multiple participants. The dynamic nature of collaborative software development, regarding the number and roles of participants as well as the scope of the project, calls for effective ways to stimulate code practices that can help increase quality in a project.

To answer this, an original methodology in addition to a visual tool's prototype, was created by this research. Both intended to support software developers to better understand code metrics information so as to produce source code with better quality. The methodology presented consists of a sequence of steps that a developer has to take as a means to having their code's software metrics extracted, stored and, through the visual tool, analyzed and displayed in a dashboard as a way to allow improvements on a project of collaborative software development, based on software quality parameters such as complexity, reusability, testability and maintainability.

In the next steps we intend to extend the types of roles addressed by the methodology, so as to allow contributions to the collaborative environment, such as better team management. Moreover, it is also possible to expand the visual tool's code analysis functionality, mainly historical quality analysis, in order to provide more information regarding the progression of quality throughout the project's development. Currently, to test the feasibility of such future directions, we are in the early stages of a full evaluation in a real-world scenario.

VIII. ACKNOWLEDGMENTS

We would like to thank Alexandre Hauße and Matheus Kevin Pelegrini for their support in the tools used for implementing the prototype. We would also like to thank Siemens Ltd for partially supporting this work.

REFERENCES

- [1] G. Vale, D. Albuquerque, E. Figueiredo, and A. Garcia, "Defining metric thresholds for software product lines: a comparative study," in *Proceedings of the 19th International Conference on Software Product Line*. ACM, 2015, pp. 176–185.
- [2] P. Vytovtov and E. Markov, "Source code quality classification based on software metrics," in *Open Innovations Association (FRUCT), 2017 20th Conference of*. IEEE, 2017, pp. 505–511.
- [3] P. R. de Bassi, G. M. P. Wanderley, P. H. Banali, and E. C. Paraiso, "Measuring developers' contribution in source code using quality metrics," in *2018 IEEE 22nd International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, 2018, pp. 39–44.
- [4] V. R. Basili, L. C. Briand, and W. L. Melo, "A validation of object-oriented design metrics as quality indicators," *IEEE Transactions on software engineering*, vol. 22, no. 10, pp. 751–761, 1996.
- [5] N. Jayalakshmi and N. Satheesh, "Software quality assessment in object based architecture," *International Journal of Computer Science and Mobile Computing*, vol. 3, no. 3, pp. 941–946, 2014.
- [6] A. S. Nuñez-Varela, H. G. Pérez-Gonzalez, F. E. Martínez-Perez, and C. Soubervielle-Montalvo, "Source code metrics: A systematic mapping study," *Journal of Systems and Software*, vol. 128, pp. 164–197, 2017.
- [7] O. Kononenko, O. Baysal, and M. W. Godfrey, "Code review quality: how developers see it," in *Software Engineering (ICSE), 2016 IEEE/ACM 38th International Conference on*. IEEE, 2016, pp. 1028–1038.
- [8] M. Alenezi, "Software architecture quality measurement stability and understandability," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 7, no. 7, pp. 550–559, 2016.
- [9] B. Ray, D. Posnett, V. Filkov, and P. Devanbu, "A large scale study of programming languages and code quality in github," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, 2014, pp. 155–165.
- [10] N. Sae-Lim, S. Hayashi, and M. Saeki, "An investigative study on how developers filter and prioritize code smells," *IEICE TRANSACTIONS on Information and Systems*, vol. 101, no. 7, pp. 1733–1742, 2018.
- [11] M.-J. Bae, J.-H. Ji, and G. Woo, "H-curve: A simple visualizing method of source code," in *Third 2008 International Conference on Convergence and Hybrid Information Technology*. IEEE, 2008, pp. 775–780.
- [12] V. Winter, C. Reinke, and J. Guerrero, "Sextant: A tool to specify and visualize software metrics for java source-code," in *Emerging Trends in Software Metrics (WETSoM), 2013 4th International Workshop on*. IEEE, 2013, pp. 49–55.
- [13] G. Balogh and A. Beszedes, "Codemetropolis: a minecraft based collaboration tool for developers," in *Software Visualization (VISUOFT), 2013 First IEEE Working Conference on*. IEEE, 2013, pp. 1–4.
- [14] S. Yu and S. Zhou, "A survey on metric of software complexity," in *Information Management and Engineering (ICIME), 2010 The 2nd IEEE International Conference on*. IEEE, 2010, pp. 352–356.
- [15] T. G. Filó, M. Bigonha, and K. Ferreira, "A catalogue of thresholds for object-oriented software metrics," *Proc. of the 1st SOFTENG*, pp. 48–55, 2015.
- [16] D. De Silva, N. Kodagoda, and H. Perera, "Applicability of three complexity metrics," in *Advances in ICT for Emerging Regions (ICTer), 2012 International Conference on*. IEEE, 2012, pp. 82–88.
- [17] H. Li, "A novel coupling metric for object-oriented software systems," in *Knowledge Acquisition and Modeling Workshop, 2008. KAM Workshop 2008. IEEE International Symposium on*. IEEE, 2008, pp. 609–612.
- [18] M. F. Oliveira, R. M. Redin, L. Carro, L. da Cunha Lamb, and F. R. Wagner, "Software quality metrics and their impact on embedded software," in *Model-based Methodologies for Pervasive and Embedded Software, 2008. MOMPES 2008. 5th International Workshop on*. IEEE, 2008, pp. 68–77.
- [19] H. M. Olague, L. H. Etzkorn, and G. W. Cox, "An entropy-based approach to assessing object-oriented software maintainability and degradation—a method and case study," in *Software Engineering Research and Practice*, 2006, pp. 442–452.
- [20] J. Anderson, "Using software tools and metrics to produce better quality test software," in *AUTOTESTCON 2004. Proceedings*. IEEE, 2004, pp. 293–297.
- [21] C. Horstman, "Big java: Programming and practice," 2002.
- [22] B. Henderson-Sellers, *Object-oriented metrics: measures of complexity*. Prentice-Hall, Inc., 1995.
- [23] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Transactions on software engineering*, vol. 20, no. 6, pp. 476–493, 1994.